MATERI LANJUTAN · ARSITEKTUR PERANGKAT LUNAK

Domain-Driven Design dalam Konteks Akademik

Menulis kode yang berbicara dengan bahasa kampus: KRS, KHS, SKS, IPK, dan yudisium.

Ubiquitous language | Bounded context | Value object & entity | Aggregate | Domain event

Apa itu Domain-Driven Design?

01

Domain di pusat

Aturan akademik adalah inti aplikasi. UI, database, dan API hanyalah detail di pinggirnya.

02

Bicara dengan pakar

Programmer, BAAK, kaprodi, dan dosen wali memakai istilah yang sama, sampai ke nama kelas.

03

Model yang kaya

Objek domain menjaga aturannya sendiri, bukan sekadar wadah data yang diolah di tempat lain.

> Diperkenalkan Eric Evans (2003). Melengkapi clean architecture: clean architecture mengatur **lapisan**, DDD mengatur **isi lapisan domain**.

PRINSIP 1

Ubiquitous language: istilah kampus = nama di kode

```
// Salah: istilah teknis, pakar domain tidak paham
class DataUser {
  List<Map> items = [];
  double calcScore() { ... }
  bool checkLimit(int n) { ... }
}

// Benar: sama dengan istilah BAAK dan dosen wali
class Krs {
  List<MataKuliah> mataKuliah = [];
  int get totalSks => ...;
  bool melebihiBatasSks(MataKuliah mk) => ...;
}
```

Istilah kampus	Di kode
Kartu Rencana Studi	`Krs`
Kartu Hasil Studi	`Khs`
Satuan Kredit Semester	`sks`
Indeks Prestasi Semester	`Ips`
Mata kuliah prasyarat	`prasyarat`
Yudisium	`bisaYudisium()`

PRINSIP 2

Bounded context: satu kata, makna berbeda

Konteks Akademik

Mahasiswa = NIM, prodi, IPK, KRS yang diambil.

KRS · KHS · Transkrip

Konteks Keuangan

Mahasiswa = NIM, golongan UKT, tagihan, status bayar.

UKT · Tagihan · Beasiswa

Konteks Kemahasiswaan

Mahasiswa = NIM, organisasi, prestasi, poin SKPI.

UKM · Prestasi · SKPI

```
// akademik/domain/mahasiswa.dart
class Mahasiswa { Nim nim; Ipk ipk; ... }
```

```
// keuangan/domain/mahasiswa.dart
class Mahasiswa { Nim nim; GolonganUkt ukt; ... }
```

> Jangan paksa satu kelas `Mahasiswa` raksasa melayani semua bagian. Yang dibagi antarkonteks cukup identitasnya: **NIM**.

BLOK BANGUNAN 1

Value object: nilai tanpa identitas, selalu valid

```
enum NilaiHuruf {
  a(4.0), ab(3.5), b(3.0), bc(2.5), c(2.0), d(1.0), e(0.0);
  const NilaiHuruf(this.bobot);
  final double bobot;
  bool get lulus => this != NilaiHuruf.e;
}

class Ipk {
  final double nilai;
  Ipk(this.nilai) {
    if (nilai < 0 || nilai > 4) throw ArgumentError('IPK 0-4');
  }
  bool get cumlaude => nilai >= 3.51;

  @override
  bool operator ==(Object o) => o is Ipk && o.nilai == nilai;
  @override
  int get hashCode => nilai.hashCode;
}
```

Tidak bisa salah: IPK 4,7 atau nilai huruf "F" tidak mungkin dibuat.

Sama bila isinya sama: dua `Ipk(3.6)` dianggap setara.

Immutable: untuk mengubah, buat nilai baru.

Contoh lain di kampus: `Sks`, `Semester`, `KodeMataKuliah`.

BLOK BANGUNAN 2

Entity: punya identitas yang bertahan

```
class Nim {                          // value object sebagai identitas
  Nim(this.nilai) {
    if (!RegExp(r'^\d{2}\.\d{2}\.\d{4}$').hasMatch(nilai)) {
      throw ArgumentError('Format NIM tidak valid');
    }
  }
  final String nilai;
  // == dan hashCode berdasarkan nilai, seperti Ipk
}

class Mahasiswa {
  Mahasiswa(this.nim, this.nama, this.programStudi);
  final Nim nim;           // identitas, tidak berubah
  String nama;             // atribut, boleh berubah
  String programStudi;

  @override
  bool operator ==(Object o) => o is Mahasiswa && o.nim == nim;
  @override
  int get hashCode => nim.hashCode;
}
```

Mahasiswa yang pindah prodi atau mengganti nama tetap **orang yang sama**, karena NIM-nya sama.

Bandingkan dengan value object: IPK 3,6 milik siapa pun tetap IPK 3,6.

Uji cepat: perlu dilacak dari waktu ke waktu? Entity. Cukup dilihat nilainya? Value object.

BLOK BANGUNAN 3

Aggregate: KRS menjaga aturannya sendiri

```
class Krs {                                  // aggregate root
  Krs(this.nim, this.semester, this.ipsLalu);
  final Nim nim;
  final String semester;
  final Ips ipsLalu;
  final _mataKuliah = <MataKuliah>[];
  bool _disetujui = false;
  int get totalSks => _mataKuliah.fold(0, (a, m) => a + m.sks);
  int get batasSks => ipsLalu.nilai >= 3.0 ? 24
      : ipsLalu.nilai >= 2.5 ? 21 : 18;
  void tambah(MataKuliah mk, Transkrip transkrip) {
    if (_disetujui) throw StateError('KRS sudah disetujui');
    if (_mataKuliah.contains(mk)) throw StateError('Sudah diambil');
    if (totalSks + mk.sks > batasSks) throw StateError('Melebihi batas');
    if (!mk.prasyarat.every(transkrip.sudahLulus)) {
      throw StateError('Prasyarat ${mk.nama} belum lulus');
    }
    _mataKuliah.add(mk);
  }
}
```

Invariant yang dijaga

Batas SKS mengikuti IPS semester lalu.

Mata kuliah tidak boleh diambil dua kali.

Prasyarat harus sudah lulus.

KRS yang disetujui dosen wali terkunci.

Dari luar, daftar mata kuliah **hanya bisa diubah lewat** `tambah()`.

BLOK BANGUNAN 4

Domain event: kabar bahwa sesuatu telah terjadi

```
class KrsDisetujui {                        // fakta, nama bentuk lampau
  const KrsDisetujui(this.nim, this.semester, this.totalSks);
  final Nim nim;
  final String semester;
  final int totalSks;
}

// di dalam aggregate Krs
final events = <Object>[];

void setujui() {
  if (_mataKuliah.isEmpty) throw StateError('KRS kosong');
  _disetujui = true;
  events.add(KrsDisetujui(nim, semester, totalSks));
}

// konteks Keuangan mendengarkan, tanpa Akademik tahu
void onKrsDisetujui(KrsDisetujui e) =>
    layananTagihan.buatTagihanSks(e.nim, e.totalSks);
```

Konteks Akademik **tidak memanggil** Keuangan secara langsung; ia hanya mengumumkan fakta.

Konteks lain bebas bereaksi: Keuangan membuat tagihan, notifikasi mengirim pesan ke mahasiswa.

Menambah reaksi baru tidak mengubah aggregate `Krs`.

BLOK BANGUNAN 5

Repository: koleksi aggregate, kontraknya milik domain

```dart
// akademik/domain/krs_repository.dart
abstract class KrsRepository {
  Future<Krs?> cari(Nim nim, String semester);
  Future<void> simpan(Krs krs);
}

// akademik/infrastructure/krs_repository_siakad.dart
class KrsRepositorySiakad implements KrsRepository {
  KrsRepositorySiakad(this._api);
  final SiakadApi _api;

  @override
  Future<Krs?> cari(Nim nim, String semester) async {
    final json = await _api.get('/krs/${nim.nilai}/$semester');
    return json == null ? null : KrsMapper.dariJson(json);
  }

  @override
  Future<void> simpan(Krs krs) =>
      _api.put('/krs/${krs.nim.nilai}', KrsMapper.keJson(krs));
}
```

Satu repository **per aggregate root**: `KrsRepository`, bukan `MataKuliahKrsRepository`.

Domain hanya mengenal kontrak; JSON, http, dan SQLite tinggal di `infrastructure`.

Mapper menerjemahkan format SIAKAD ke model domain, sehingga perubahan API tidak merembes.

BLOK BANGUNAN 6

Domain service: aturan yang bukan milik satu objek

```
class PenilaiYudisium {
  static const minSks = 144;
  static const minIpk = 2.0;

  HasilYudisium nilai(Transkrip t, List<Skripsi> skripsi) {
    final alasan = <String>[];
    if (t.totalSks < minSks) alasan.add('SKS kurang dari $minSks');
    if (t.adaNilai(NilaiHuruf.e)) alasan.add('Masih ada nilai E');
    if (t.ipk.nilai < minIpk) alasan.add('IPK di bawah $minIpk');
    if (!skripsi.any((s) => s.lulusSidang)) {
      alasan.add('Skripsi belum lulus sidang');
    }
    return alasan.isEmpty
        ? HasilYudisium.lulus(predikat: t.ipk.cumlaude
            ? 'Cumlaude' : 'Sangat Memuaskan')
        : HasilYudisium.tunda(alasan);
  }
}
```

Keputusan yudisium butuh **transkrip dan skripsi** sekaligus, jadi tidak cocok ditaruh di salah satunya.

Domain service tetap Dart murni, tanpa state, dan mudah diuji.

Namanya kata kerja atau peran dari bahasa kampus: *penilai yudisium*.

BLOK BANGUNAN 7

Application service: merangkai satu use case

```dart
// akademik/application/ajukan_mata_kuliah.dart
class AjukanMataKuliah {
  AjukanMataKuliah(this._krsRepo, this._transkripRepo);
  final KrsRepository _krsRepo;
  final TranskripRepository _transkripRepo;

  Future<void> call(Nim nim, String semester, MataKuliah mk) async {
    final krs = await _krsRepo.cari(nim, semester)
        ?? (throw StateError('Masa KRS belum dibuka'));
    final transkrip = await _transkripRepo.milik(nim);

    krs.tambah(mk, transkrip);            // aturan dijaga aggregate
    await _krsRepo.simpan(krs);
  }
}
```

Tugasnya hanya **mengatur urutan**: ambil, minta aggregate bekerja, simpan.

Tidak ada `if` aturan akademik di sini. Aturan tetap di `Krs`.

Satu kelas per use case: `AjukanMataKuliah`, `SetujuiKrs`, `AjukanYudisium`.

Struktur folder Flutter per bounded context

```
lib/
  akademik/                    // bounded context
    domain/
      krs.dart                 // aggregate root
      mata_kuliah.dart         // entity
      nilai_huruf.dart  ipk.dart  nim.dart   // value object
      krs_disetujui.dart       // domain event
      penilai_yudisium.dart    // domain service
      krs_repository.dart      // kontrak repository
    application/
      ajukan_mata_kuliah.dart  // use case
    infrastructure/
      krs_repository_siakad.dart
    presentation/
      krs_view_model.dart  halaman_krs.dart
  keuangan/                    // bounded context lain
    domain/  application/  infrastructure/  presentation/
```

Folder dikelompokkan **per konteks bisnis** dulu, baru per lapisan.

Folder `domain/` tidak boleh mengimpor Flutter, http, maupun konteks lain.

Tim Keuangan bisa bekerja di `keuangan/` tanpa menyentuh `akademik/`.

Bertemu MVVM: ViewModel memanggil use case

```
class KrsViewModel extends ChangeNotifier {
  KrsViewModel(this._ajukan, this.nim, this.semester);
  final AjukanMataKuliah _ajukan;
  final Nim nim;
  final String semester;
  String? pesan;

  Future<void> pilih(MataKuliah mk) async {
    try {
      await _ajukan(nim, semester, mk);
      pesan = '${mk.nama} ditambahkan ke KRS';
    } on StateError catch (e) {
      pesan = e.message;                  // pesan aturan dari domain
    }
    notifyListeners();
  }
}
```

View → ViewModel → Use case → Aggregate

ViewModel tidak tahu batas 24 SKS atau aturan prasyarat.

Bila kampus mengubah batas SKS, yang berubah hanya `Krs.batasSks`; layar ikut menampilkan pesan barunya.

Anti-pattern: anemic domain model

Anemic: aturan tersebar

```
class Krs { List<MataKuliah> daftar = []; }

// di ViewModel A
if (krs.daftar.fold(0, (a, m) => a + m.sks)
    + mk.sks <= 24) krs.daftar.add(mk);

// di ViewModel B (lupa cek prasyarat)
krs.daftar.add(mk);
```

Rich: aturan di satu tempat

```
class Krs {
  final _mataKuliah = <MataKuliah>[];
  void tambah(MataKuliah mk, Transkrip t) {
    // semua invariant dicek di sini
  }
}

// di mana pun
krs.tambah(mk, transkrip);
```

> Gejala anemic: kelas domain hanya berisi field dan getter, sementara `if` aturan akademik muncul berulang di ViewModel dan widget.

Ringkasan

Konsep	Contoh akademik
Ubiquitous language	Krs, Khs, Sks, Ips
Bounded context	Akademik, Keuangan
Value object	NilaiHuruf, Ipk, Nim
Entity	Mahasiswa, MataKuliah
Aggregate	Krs dan invariant-nya
Domain event	KrsDisetujui
Domain service	PenilaiYudisium
Application service	AjukanMataKuliah

Checkpoint HOTS

C4 · Klasifikasikan Semester, Dosen, dan Jadwal Kuliah: entity atau value object? Jelaskan.

C5 · Aturan "maksimal 2 mata kuliah mengulang per semester" sebaiknya dijaga di Krs atau di domain service? Nilailah.

C6 · Rancang aggregate Bimbingan Skripsi: invariant, domain event, dan repository-nya.